Image compression using Jupyter Notebook

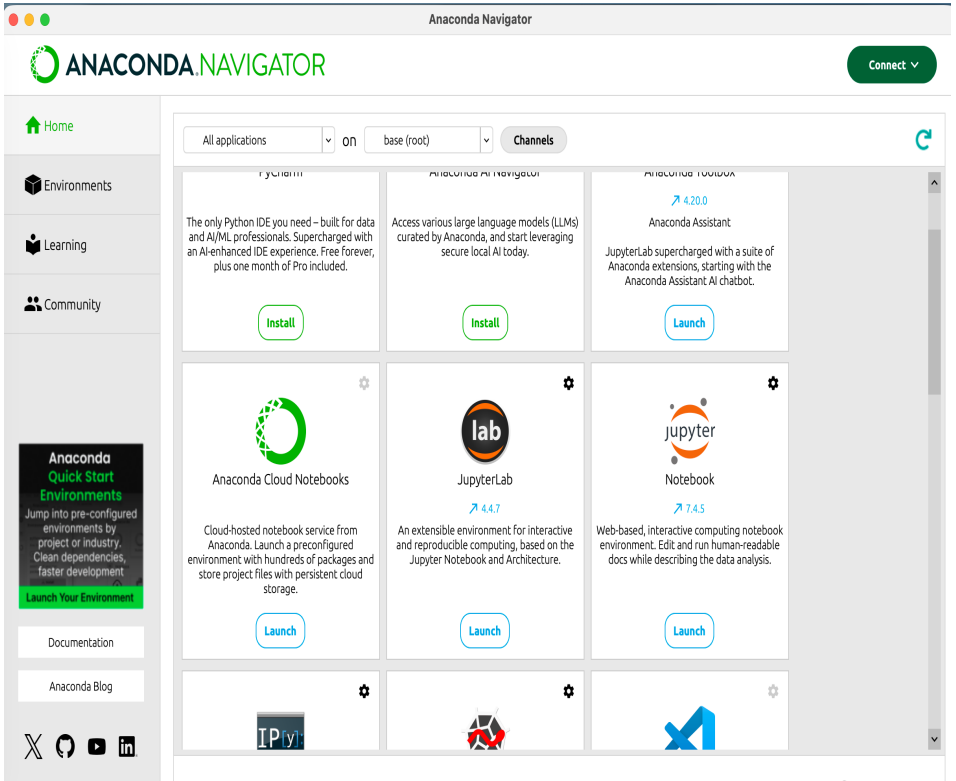
Lab Session 1

Overview Lab Session

- Anaconda Navigator & Jupyter Notebook
- Baseline YOLO detection for image &Image Compression
- Task 1
- Object Detection Vs ROI
- Task 2

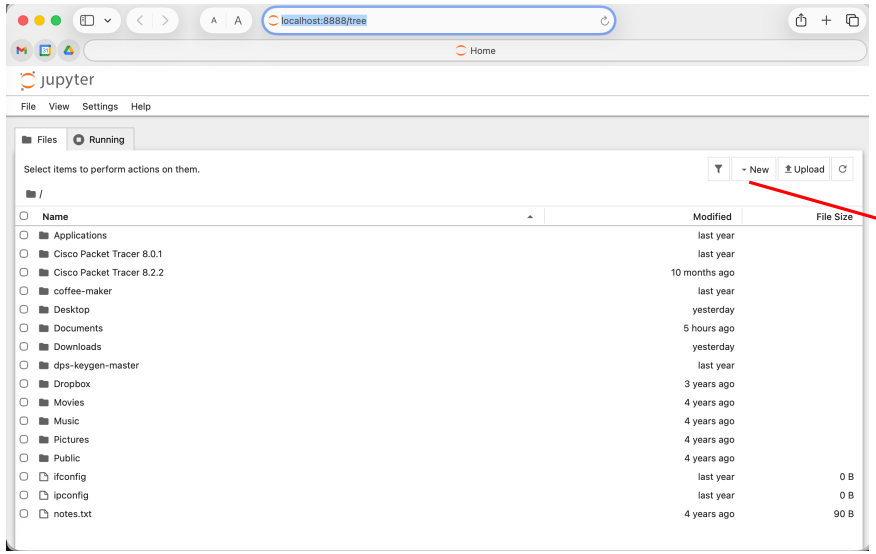
Launch Anaconda Navigator

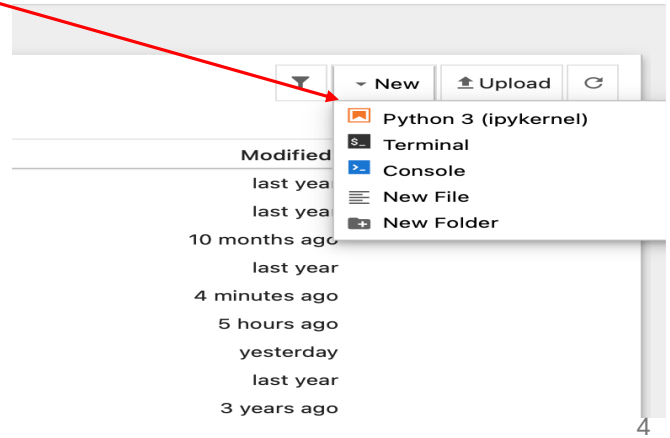
- In the Home Tab, locate Jupyter Notebook tile.
- Click the Lauch button under the tile
- This will automatically start a local server and open the Jupyter Notebook Dashboard in your default web browser.



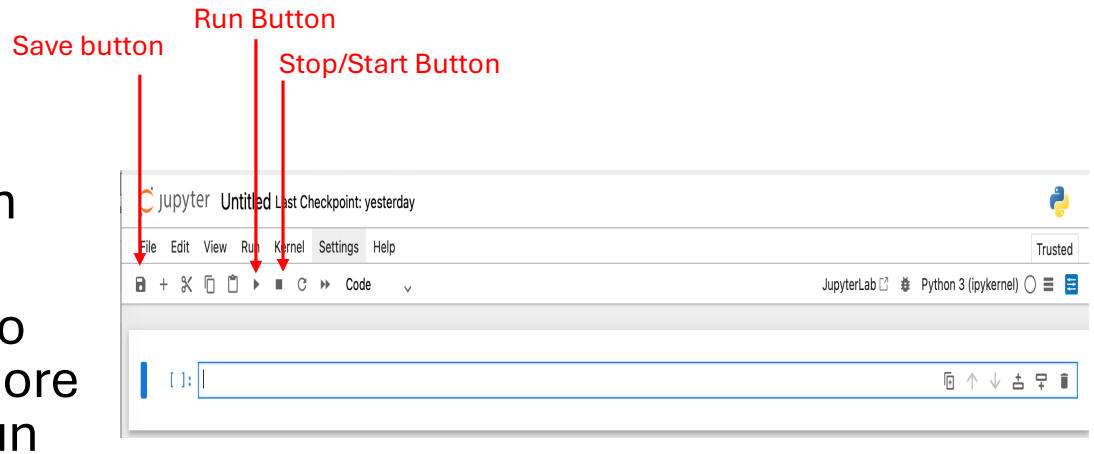
Open Notebook

- To create a new notebook, in the top-right corner of the browser dashboard, click New and Select Phyhon 3
- To open an existing notebook, navigate through the file explorer in the browser and click on any file with the .ipynb extension





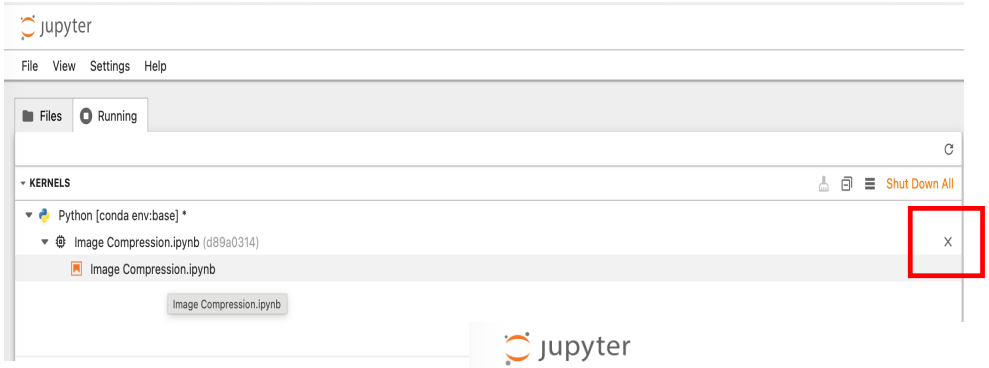
Basic Operation

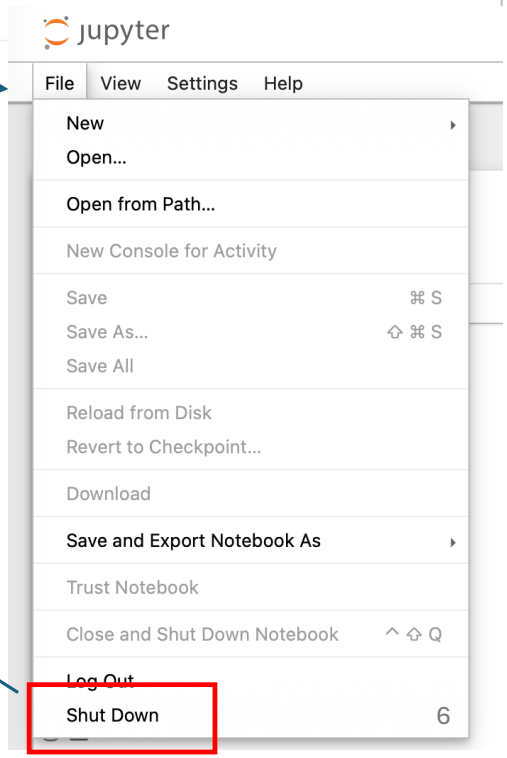


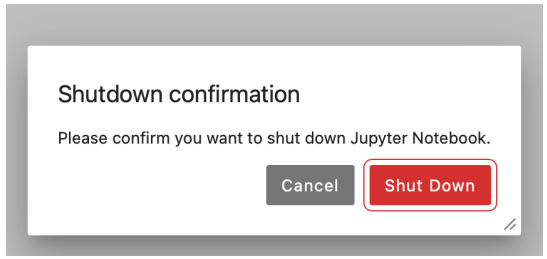
- Write a Code- type your Python code into a cell
- Run a cell- Press Shift +Enter to execute the current cell and more to the next one , or click the Run button in the toolbar
- Save Work – Click the Save icon or press Ctrl + 5
- Stop/Restart - Use the kernel menu to 'Interupt' a log-running process or Restart the session if it becomes unresponsive

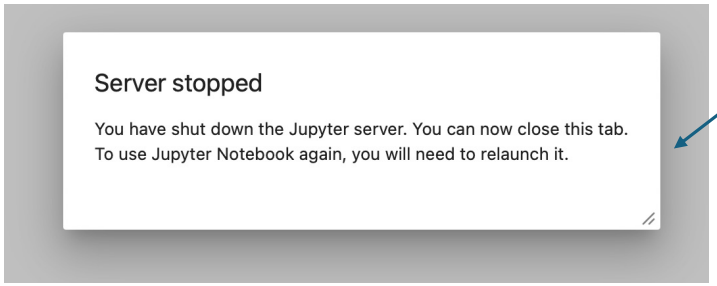
Properly Close

- Close the Notebook- go to File -> Close and Halt to stop the kernel for that specific file
- Close the Server – Closing the browser tab does not stop Jupyter. To fully shut it down, return to the Jupyter Dashboard and click File Tab in the top-left corner, click Shut down,



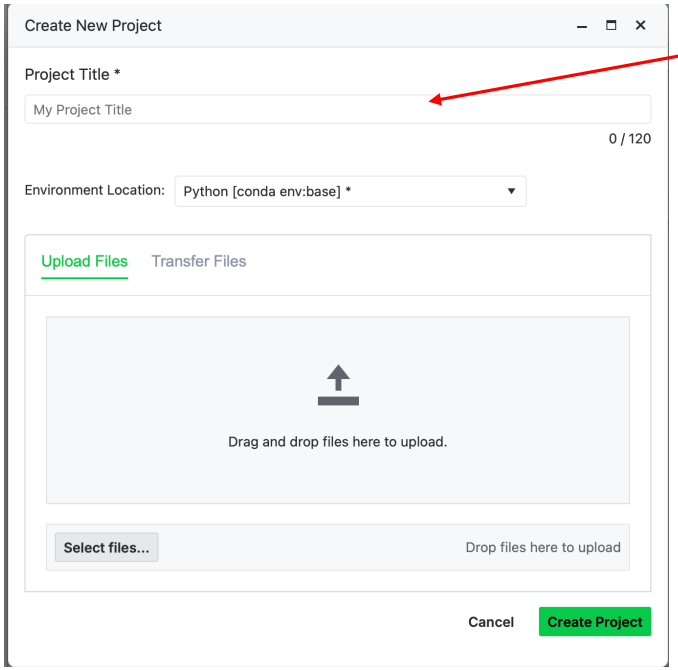


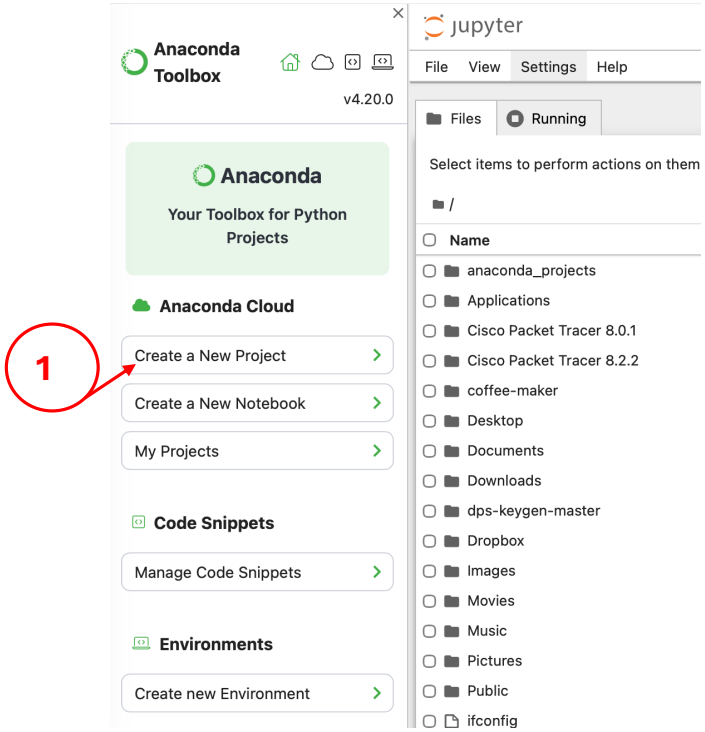




Project Management in Anaconda/Jupyter

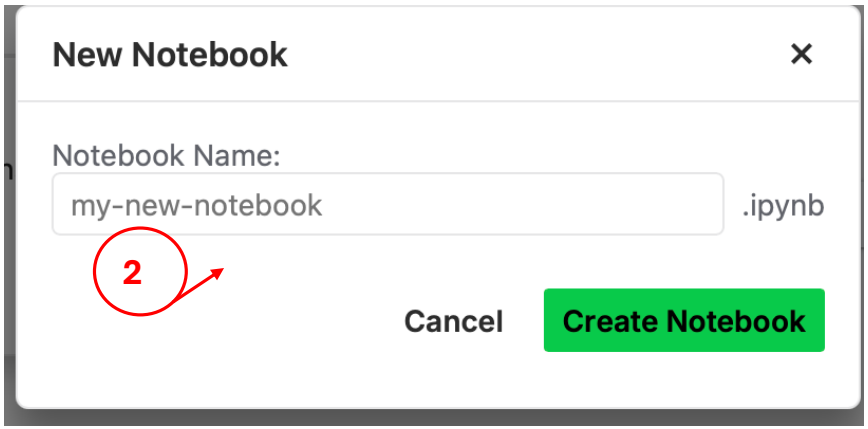
- Good to start any project by creating new project folder

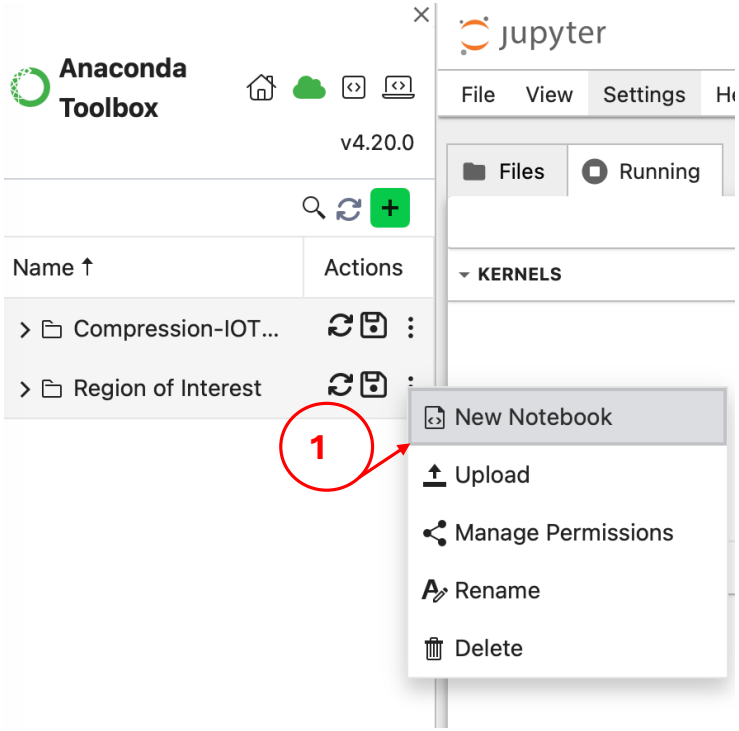




Creating new notebook in your project folder

- In your project folder add new notebook to start your work





Install and import:

- In a code cell copy and paste the python code
- Click the run button in the cell
- Wait until the downloading finish

```
[*]: !pip install ultralytics opencv-python matplotlib

from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import os
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.39-py3-none-any.whl.metadata (39 kB)
Collecting opencv-python
  Downloading opencv_python-4.13.0.92-cp37-abi3-macosx_13_0_arm64.whl.metadata (19 kB)
Requirement already satisfied: matplotlib in /opt/anaconda3/lib/python3.13/site-packages (3.10.6)
Requirement already satisfied: numpy>=1.23.0 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (2.3.5)
Requirement already satisfied: pillow>=7.1.2 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (12.0.0)
Requirement already satisfied: pyyaml>=5.3.1 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (2.32.5)
Requirement already satisfied: scipy>=1.4.1 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (1.16.3)
Collecting torch>=1.8.0 (from ultralytics)
  Downloading torch-2.11.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (29 kB)
Collecting torchvision>=0.9.0 (from ultralytics)
  Downloading torchvision-0.26.0-cp313-cp313-macosx_12_0_arm64.whl.metadata (5.5 kB)
Requirement already satisfied: psutil>=5.8.0 in /opt/anaconda3/lib/python3.13/site-packages (from ultralytics) (7.0.0)
Collecting polars>=0.20.0 (from ultralytics)
  Downloading polars-1.40.0-py3-none-any.whl.metadata (10 kB)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
```

< >Python code

```
!pip install ultralytics opencv-python matplotlib

from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import os
```

outcome

```
Successfully installed opencv-python-4.13.0.92 polars-1.40.0 polars-runtime-32-1.40.0 torch-2.11.0 torchvision-0.26.0 ultralytics-8.4.39 ult
ralytics-thop-2.0.18
```

Load sample image

- Before running this code, you need to download some .jpg image in the same folder of your .ipyb file
- Look for some sample .jpg images

```
[10]: (np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))
```

Original Image

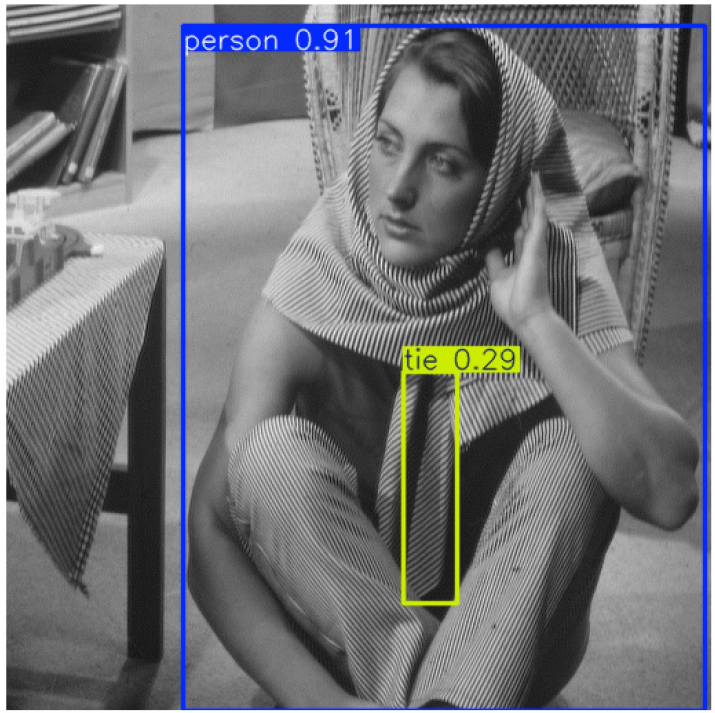
< >Python code

```
image_path = ‘ barbara.jpg’  # provide test images

img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
```

Baseline of YOLO Detection for image



< >Python code

```
model = YOLO('yolov8n.pt')  # lightweight model
results = model(image_path)
results[0].show()
```

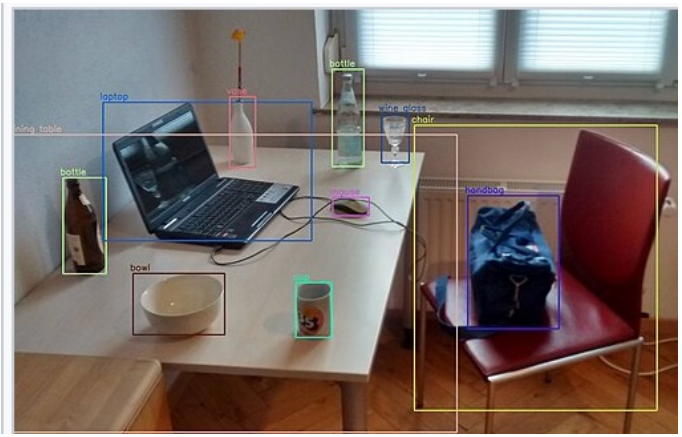
Record:

1. Number of objects detected
2. Confidence scores

Add more images and observe

- Add images in the **image_path**
- What can you observe with different images?

YOLO Detection



The concept of YOLO:
Object detection pipeline
Bounding boxes + confidence

- YOLO model is used to detection images
- You Only Look Once (YOLO) is a series of real-time object detection systems based on convolutional neural networks.
- First introduced by Joseph Redmon et al. in 2015, YOLO has undergone several iterations and improvements
- Becoming one of the most popular object detection frameworks.

Image compress

< >Python code

```
def compress_image(input_path, output_path, quality):
    img = cv2.imread(input_path)
    cv2.imwrite(output_path, img, [int(cv2.IMWRITE_JPEG_QUALITY), quality])
```

< >Python code

```
qualities = [90, 70, 50, 30, 10] #different quality for the compressed image
compressed_paths = []
for q in qualities:
    out_path = f'compressed_{q}.jpg'
    compress_image(image_path, out_path, q)
    compressed_paths.append(out_path)
```

Generate visual comparison

< >Python code

```
plt.figure(figsize=(10,5))

for i, path in enumerate(compressed_paths):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.subplot(1,5,i+1)
    plt.imshow(img)
    plt.title(f"Q={qualities[i]}")
    plt.axis('off')
```

< >Python code

```
detection_results = {}

for path in compressed_paths:
    results = model(path)
    detection_results[path] = results
    results[0].show()
```

File size code

< >Python code

```
import os
for path in compressed_paths:
    size = os.path.getsize(path)/1024
    print(path, ":", round(size,2), "KB")
```

Expected outcome

```
compressed_90.jpg : 2414.34 KB
compressed_70.jpg : 1139.0 KB
compressed_50.jpg : 808.26 KB
compressed_30.jpg : 585.56 KB
compressed_10.jpg : 341.81 KB
```

Observe the outcome of the different pictures

Different Resolution and YOLO model

Latest and most advanced

- YOLO26 (Jan 2026)-current cutting-edge model
- YOLO11 (Sept 2024) – highly efficient fast and accurate
- YOLOE – A 2025 model focused on an open vocabulary detection, allowing it to detect custom, unseen classes via text or image prompts without retraining.
- YOLOv12 (Feb 2025) – Focused on attention-centric detection, integrating Area Attention and Flash Attention into CNN structure to improve accuracy in high-velocity complex environments

Stable and popular

- YOLO8 (2023) – A very popular, versatile model offering a good balance of speed and accuracy, supporting detection segmentation and pose.
- YOLO9 (2024) – Introduced Programmable Gradient Information (PGI) to reduce information loss in deep networks
- YOLO10(2024) – Focused on efficiency by removing the need for NMS post processing, providing faster inference times.
- YOLO5 (2020) – A popular, well-documented model, known for stability and ease of deployment.

Lab Task

1. To run the code using the same picture with different resolution
2. To run the code using different YOLO model

Object detection Vs RoI

	OD	ROI
Aim	Locates objects (bounding boxes) and classifies them	Defines a specific sub-region to ignore background noise and save computing power.
Method	Scans the whole image (e.g., YOLO) or uses region proposals (e.g., R-CNN).	Masking, cropping, or identifying likely areas where objects exist.
Output	Object labels + bounding box coordinates	A cropped subset of the image or a specific image area for further processing

Object detection is the complete process of identifying and locating objects in an image, while a Region of Interest (ROI) is a specific, restricted area within an image chosen for analysis to speed up processing and improve efficiency.

Key difference:
Scope: Detection finds *what* and *where* everywhere. ROI limits *where* the model looks.
Efficiency: Using ROIs increases processing speed by limiting the analysis area.
Workflow: ROI is often a step *within* object detection, particularly in region-based methods like Fast R-CNN, where Region of Interest (ROI) Pooling is used to extract features

Why do we need ROI

- ROI is the focus of the important part of image.
- ROI is needed to reduce computational cost and improve detection accuracy
- It also reduces the bandwidth during transmission in the edge network

< > Python

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('people2.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
```

Original Image

Crop ROI using Pixel coordinate (Manual)

- What happen if the coordinate is wrong?
- How does ROI size affect detail?

Example of different ROI:

```
roi1 = img_rgb[50:200, 100:300]
roi2 = img_rgb[200:400, 300:500]
```

< > Python

```
# Define ROI (y1:y2, x1:x2)
roi = img_rgb[100:300, 200:400]

plt.imshow(roi)
plt.title("Manual ROI")
plt.axis('off')
```

Detecting ROI with different approaches

Using OpenCV

< > Python

```
import cv2

x, y, w, h = 200, 100, 200, 200
roi2 = img_rgb[y:y+h, x:x+w]

# Draw rectangle
img_copy = img_rgb.copy()
cv2.rectangle(img_copy, (x,y), (x+w,y+h),
(255,0,0), 2)

plt.imshow(img_copy)
plt.title("ROI Bounding Box")
plt.axis('off')
```

Using YOLO

< > Python

```
#use YOLO to detect and extract ROI
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model("people1.jpg")

for r in results:
   for box in r.boxes:
      x1, y1, x2, y2 = map(int, box.xyxy[0])
      roi2 = img_rgb[y1:y2, x1:x2]

      plt.imshow(roi)
      plt.title("Detected ROI")
      plt.axis('off')
      plt.show()
```

Processing Time - Use this example code

```
import time
import cv2

# Full image processing
start = time.time()
img = cv2.imread('people1.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray_full = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
end = time.time()
print("Full Image Time:", end - start)

# ROI processing
x, y, w, h = 200, 100, 200, 200
roi2 = img_rgb[y:y+h, x:x+w]
start = time.time()
gray_roi = cv2.cvtColor(roi2, cv2.COLOR_BGR2GRAY)
end = time.time()
print("ROI Time:", end - start)
```

Task 2

1. Compare the two approaches using OpenCV and YOLO, and analyse your observation.